Git Commands

A Visual Guide

LEARN BY SEEING

Every command, every flow, illustrated
From your laptop to GitHub and back
Based on real MediBook journey

Commands Covered Visually

Mental model: Local vs Remote

git init - initialize repository

git add - stage changes

git status - check state

git commit - save snapshot

git remote - connect to GitHub

git push - upload to GitHub

git pull - download from GitHub

git branch - manage branches

git checkout - switch branches

git merge - combine branches

git rm - remove files (with safety!)

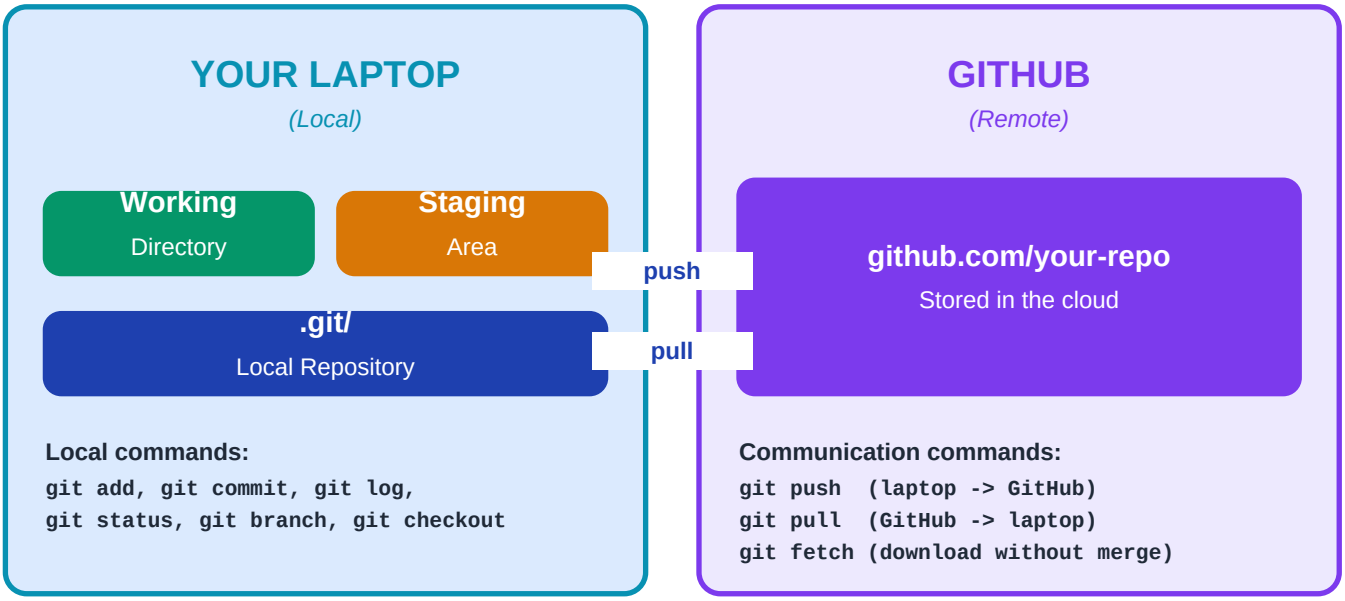
git log - view history

Complete workflow (all 10 steps)

0. The Mental Model

Before diving into commands, understand the BIG picture: Git works between two worlds - your laptop (local) and GitHub (remote).

The Two Worlds: Your Laptop vs GitHub



KEY TAKEAWAY:

Local commands stay on your laptop until you push.
GitHub knows nothing about your work until you push.
Pull brings GitHub's latest to your laptop.

1. git init

```
git init
```

Turn a regular folder into a Git repository.

The .git/ folder is hidden. It stores all Git data: history, branches, configs.

Never edit files inside .git/ manually - you can corrupt your repo.

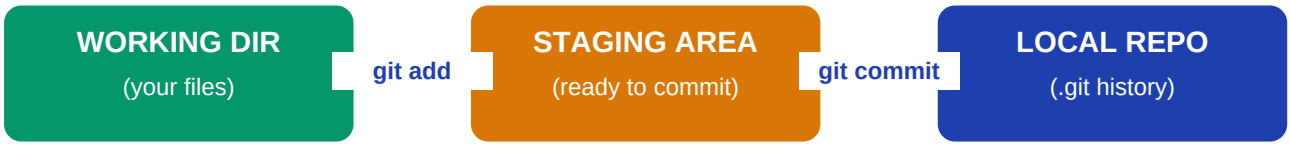
When to use:

- Starting a brand new project
- Adding an existing folder to Git for the first time
- Not needed when cloning - that auto-initializes

2. git add

```
git add .
```

Move changes from working directory to staging area (ready to commit).



Common forms:

`git add .`	->	Stage all files in current folder
`git add file.java`	->	Stage one specific file
`git add src/`	->	Stage everything in src folder
`git add *.java`	->	Stage all .java files
`git add -p`	->	Interactive - pick which changes to stage

Think of staging like a shopping cart:

You add items (changes) to the cart (staging area). Then you check out (commit). You can review what is in the cart before committing.

3. git status

`git status`

Your most-used command. Shows what's changed, staged, and untracked.

What git status shows:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  new file:   DoctorStatus.java
  modified:   SecurityConfig.java

Changes not staged for commit:
  modified:   README.md

Untracked files:
  newfile.txt
```

Color meanings:

GREEN = staged (will be committed)
RED = modified or untracked (won't be committed yet)

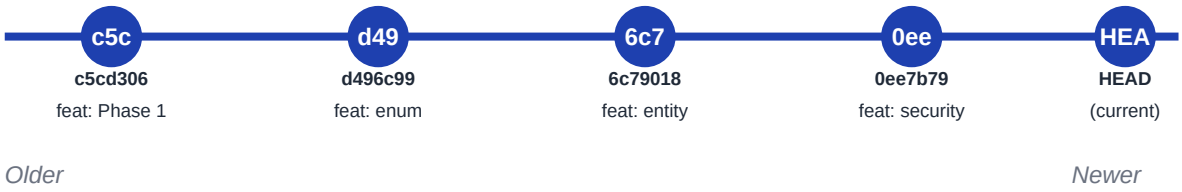
Run this constantly! Before every add, before every commit, before every push. It's free, fast, and tells you exactly where you stand.

4. git commit

```
git commit -m "your message"
```

Save a permanent snapshot of staged changes.

Each commit is a snapshot in time:



Anatomy of a commit:

```
commit c5cd306abc...                    <- SHA-1 hash (unique ID)
Author: Vijay <vijay@example.com>       <- WHO made it
Date:   Mon Jun 3 10:30:00 2026        <- WHEN

    feat: complete Phase 1              <- WHAT (the message)

    - Add User entity                   <- (body, optional)
    - Add JWT authentication
```

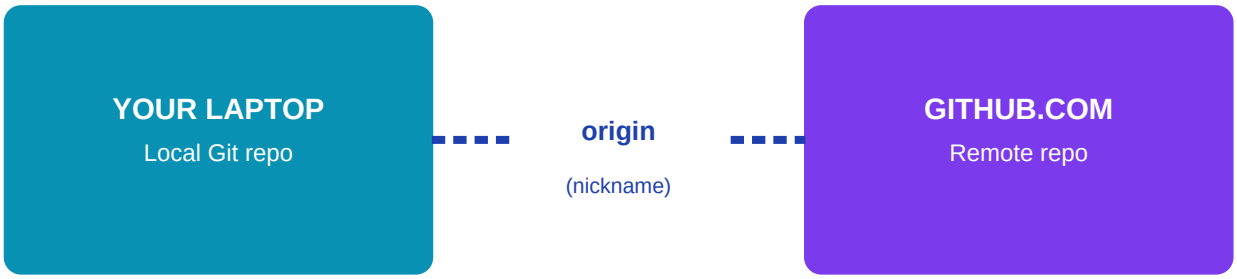
Conventional Commit Format:

feat(scope): description — New feature
fix(scope): description — Bug fix
docs(scope): description — Documentation
refactor(scope): description — Code restructure

5. git remote

`git remote add origin URL`

Connect your local repo to a remote repository (like GitHub).



Commands:

`git remote -v`	-> List all remotes
`git remote add origin URL`	-> Add new remote called 'origin'
`git remote remove origin`	-> Disconnect from a remote
`git remote set-url origin NEW_URL`	-> Change URL of existing remote

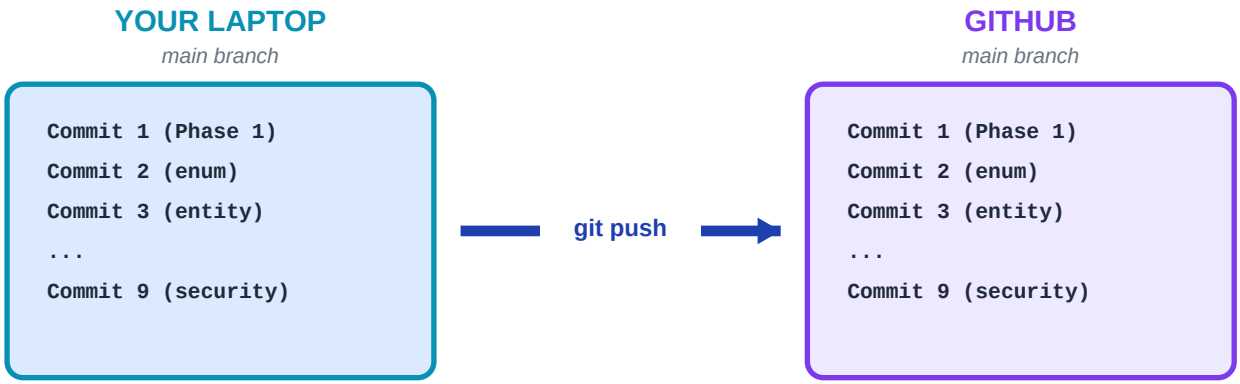
Why 'origin'?

It's just a nickname - a convention. You could call it 'github' or 'main-repo' or anything. Using 'origin' is the universal standard everyone recognizes.

6. git push

```
git push -u origin main
```

Upload your local commits to GitHub.



Common forms:

`git push`	->	Push current branch to its tracking remote
`git push origin main`	->	Push 'main' branch to 'origin' remote
`git push -u origin main`	->	First push - set upstream tracking
`git push --force`	->	Overwrite remote (DANGEROUS - avoid!)

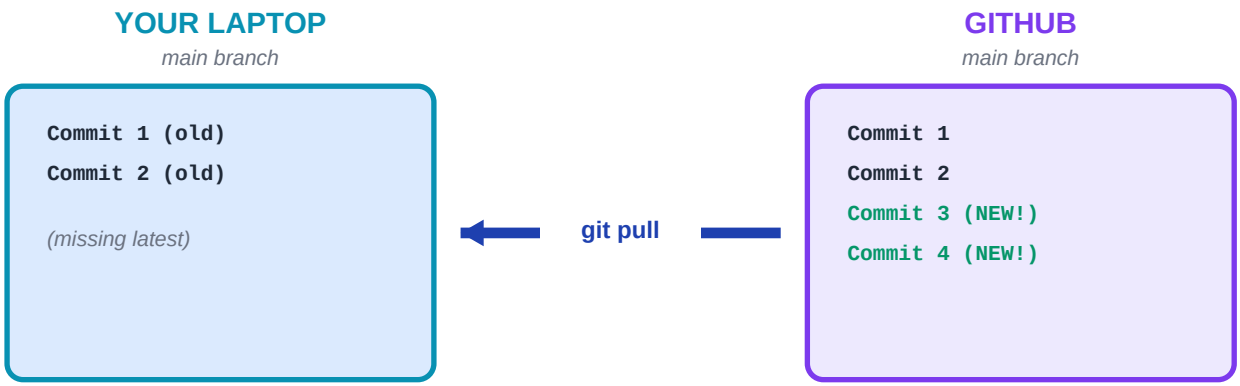
The -u flag:

Sets upstream tracking. After the first push with -u, you can just type 'git push' alone - Git remembers where to send it.

7. git pull

```
git pull origin main
```

Download GitHub's latest commits into your local branch.



What pull does internally:

git pull = git fetch + git merge

1. fetch: Downloads latest commits from remote
2. merge: Combines remote changes into your local branch

WARNING: If you have local uncommitted changes, pull might create merge conflicts. Commit or stash first!

Pull = Fetch + Merge:

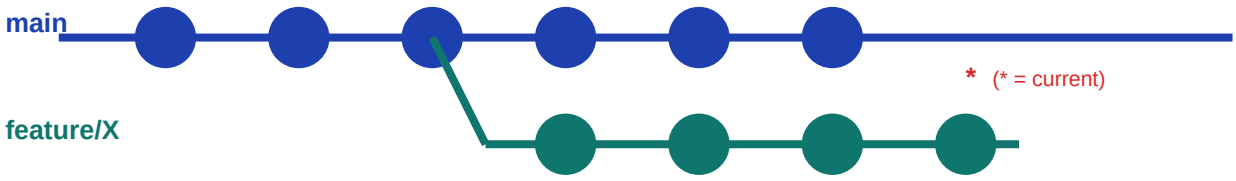
git pull is actually two commands rolled into one. If you want to inspect changes before merging, use 'git fetch' alone, review with 'git log origin/main', then 'git merge' manually.

8. git branch

`git branch -a`

List, create, and delete branches.

Branches are parallel timelines:



Common commands:

`git branch`	->	List all local branches (* = current)
`git branch -a`	->	List local + remote branches
`git branch new-feature`	->	Create new branch (don't switch)
`git branch -d branch-name`	->	Delete merged branch (safe)
`git branch -D branch-name`	->	FORCE delete branch (loses unmerged!)
`git branch -m old new`	->	Rename a branch

Branch naming:

Use prefixes for clarity: **feature/X**, **fix/X**, **hotfix/X**, **docs/X**, **chore/X**. Use kebab-case (hyphens), not camelCase.

9. git checkout

`git checkout branch-name`

Switch between branches, or create new ones.

git checkout = teleport between branches/commits

BEFORE: on main

`git checkout feature`

AFTER: on feature

Common forms:

`git checkout main`	->	Switch to existing branch 'main'
`git checkout -b new-branch`	->	CREATE + switch to new branch
`git checkout file.txt`	->	Discard local changes to a file
`git checkout abc123`	->	Move to specific commit (read-only)

MODERN ALTERNATIVE (Git 2.23+):

```
git switch main          (switch branches)
git switch -c new-branch (create + switch)
```

The confusing command:

checkout does too many things - that is why Git 2.23 split it into 'switch' (branches) and 'restore' (files). Both are still valid; use whichever feels clearer.

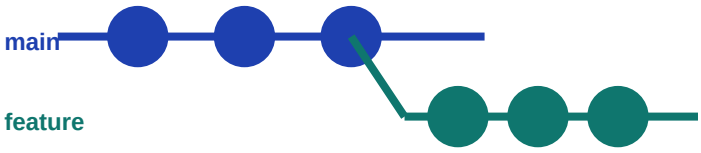
10. git merge

```
git merge feature/phase-2
```

Combine commits from another branch into your current branch.

git merge brings commits from another branch into yours:

BEFORE merge:



AFTER (fast-forward):

Two types of merges:

1. FAST-FORWARD (clean!)

When main has no new commits since branch diverged.
Git just moves main forward. Linear history.

2. THREE-WAY MERGE (creates merge commit)

When BOTH branches have new commits.
Git creates a special merge commit. History shows the split.

MERGE CONFLICT: when both branches change the same line!

You manually resolve, then 'git add' + 'git commit' to complete.

What we did in MediBook:

Phase 2 was developed on a feature branch. After testing, we merged it into main. It was a fast-forward merge because main hadn't changed since the branch was created.

11. git rm

```
git rm --cached file.txt
```

Remove files - with critical safety differences!

git rm has TWO modes - know the difference!

git rm file.txt

Effect:

X Remove from staging
X DELETE from disk!

FILE IS GONE!

git rm --cached file.txt

Effect:

+ Remove from staging
+ KEEP on disk safely

FILE IS SAFE!

Common forms:

`git rm file.txt`	->	DELETE from staging + disk
`git rm --cached file.txt`	->	Untrack file (keep on disk)
`git rm -r folder/`	->	Recursively delete folder + contents
`git rm -rf --cached .`	->	Untrack everything (we used this!)
`git rm --dry-run file.txt`	->	Preview what would happen

REAL EXAMPLE FROM MEDIBOOK:

We used 'git rm -rf --cached .' to unstage everything
without losing files when fixing the embedded git issue.

The --cached safety flag:

Without --cached, git rm DELETES the file. With --cached, it only UNSTAGES, keeping the file safe. When in doubt, ALWAYS use --cached!

12. git log

`git log --oneline`

View the history of commits.

git log shows your commit history (newest first):

```
$ git log --oneline

0ee7b79     feat(security): allow public access to GET /doctors
0923ca3     feat(doctor): add doctor, admin, public controllers
7b1e883     feat(doctor): add DoctorService with workflow logic
344855e     feat(doctor): add DoctorMapper for entity-DTO
68bb334     feat(doctor): add request and response DTOs
f0257df     feat(doctor): add DoctorRepository
6c79018     feat(doctor): add Doctor entity
d496c99     feat(doctor): add DoctorStatus enum
c5cd306     feat: complete Phase 1 - authentication module
```

Useful variations:

`git log`	->	Full history with author, date, message
`git log --oneline`	->	Compact - one line per commit
`git log --graph`	->	ASCII art branch visualization
`git log --graph --all --oneline`	->	Beautiful tree view of all branches
`git log -n 5`	->	Show only last 5 commits
`git log --author=Vijay`	->	Filter by author

13. The Complete Workflow

All commands in sequence - exactly what we did to ship Phase 2:

OUR COMPLETE PHASE 2 WORKFLOW (the journey we just did):

1	`git checkout main`	->	Start clean on main
2	`git checkout -b feature/phase-2`	->	Create feature branch
3	`git add file.java`	->	Stage each file
4	`git commit -m 'feat: ...'`	->	Commit (repeat 9 times)
5	`git push -u origin feature/phase-2`	->	Push feature branch
6	`git checkout main`	->	Back to main
7	`git merge feature/phase-2`	->	Merge feature into main
8	`git push origin main`	->	Push merged main
9	`git branch -d feature/phase-2`	->	Delete local feature
10	`git push origin --delete feature/phase-2`	->	Delete remote feature

Repeat for every new feature. This is THE professional Git workflow!

Quick Reference Cheat Sheet

Goal	Command
See what changed	`git status`
Stage all changes	`git add .`
Stage specific file	`git add file.java`
Unstage (keep on disk)	`git rm --cached file.java`
Commit changes	`git commit -m "feat: ..."`
See history	`git log --oneline`
List branches	`git branch`
Create + switch branch	`git checkout -b feature/X`
Switch branch	`git checkout main`
Merge branch into current	`git merge feature/X`
Upload to GitHub	`git push`
Download from GitHub	`git pull`
First push of new branch	`git push -u origin branch-name`
Delete local branch	`git branch -d branch-name`
Delete remote branch	`git push origin --delete branch-name`
See remotes	`git remote -v`
Add remote	`git remote add origin URL`
Undo unstaged changes	`git restore file.java`
Undo last commit (keep changes)	`git reset --soft HEAD~1`